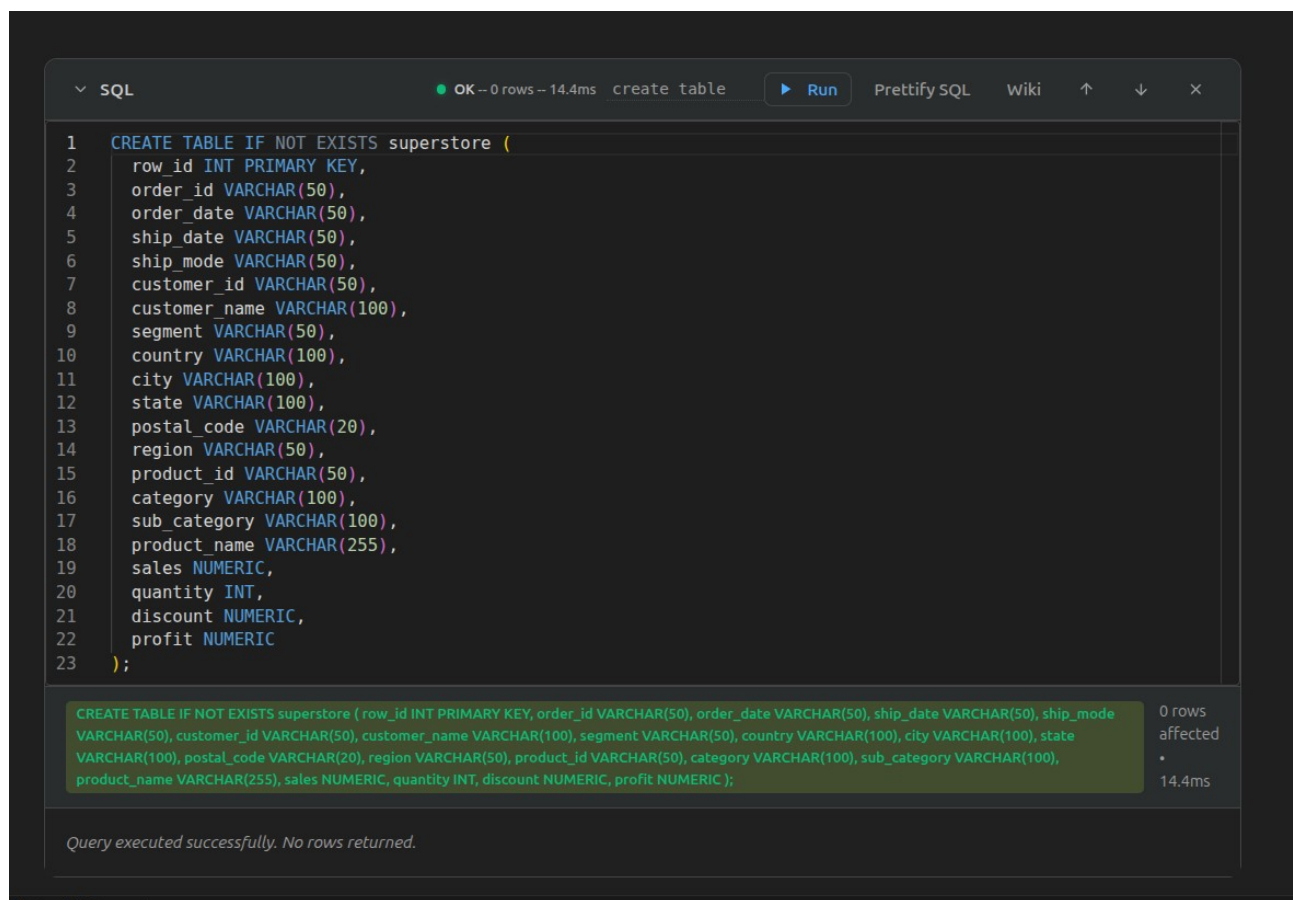


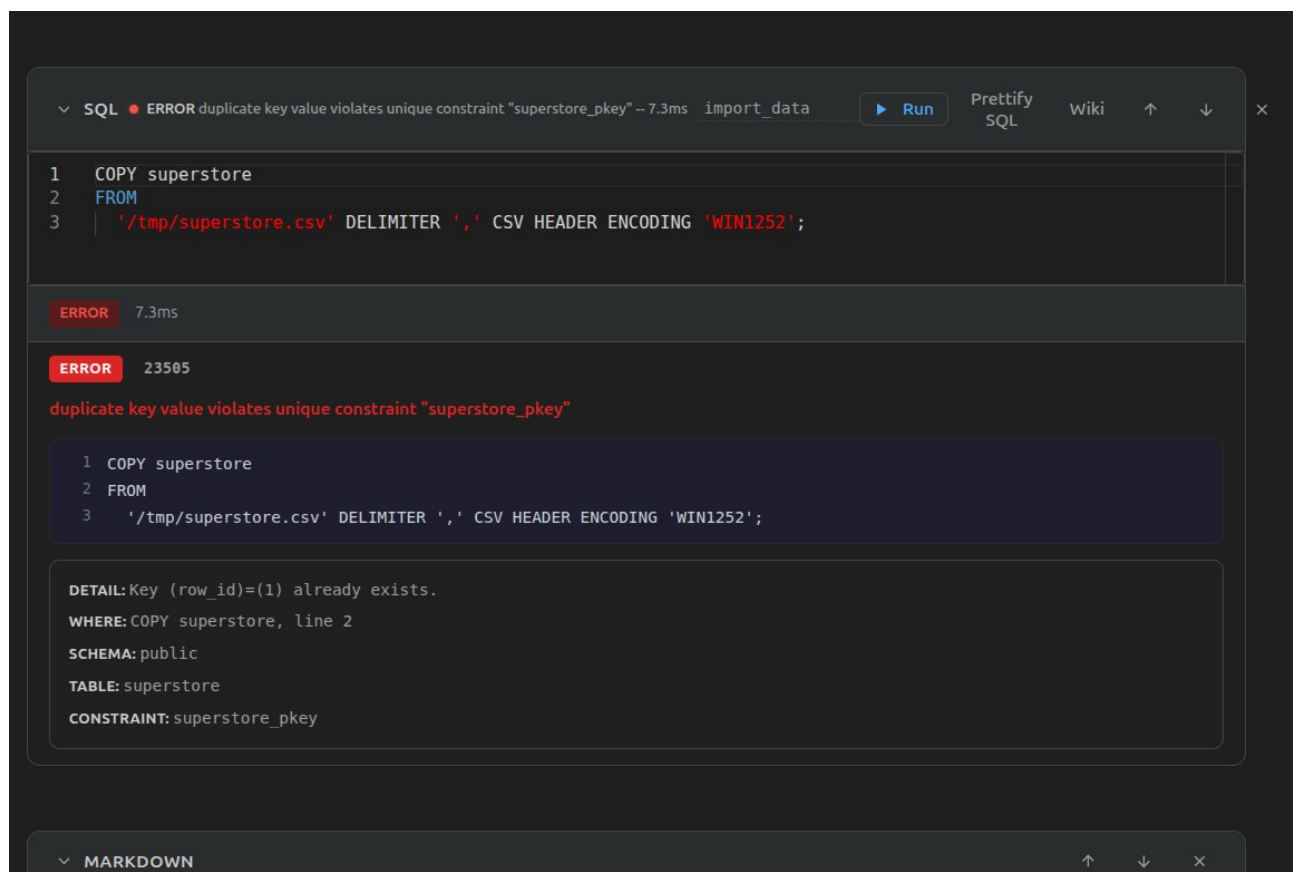
Step-1: Creating table and importing data:



The screenshot shows a SQL IDE window with a dark theme. The top toolbar includes a dropdown menu set to 'SQL', a status indicator 'OK -- 0 rows -- 14.4ms', a button 'create table', a 'Run' button, and links for 'Prettify SQL', 'Wiki', and navigation arrows. The main editor contains a SQL script to create a table named 'superstore' with various columns and data types. The script is as follows:

```
1 CREATE TABLE IF NOT EXISTS superstore (  
2     row_id INT PRIMARY KEY,  
3     order_id VARCHAR(50),  
4     order_date VARCHAR(50),  
5     ship_date VARCHAR(50),  
6     ship_mode VARCHAR(50),  
7     customer_id VARCHAR(50),  
8     customer_name VARCHAR(100),  
9     segment VARCHAR(50),  
10    country VARCHAR(100),  
11    city VARCHAR(100),  
12    state VARCHAR(100),  
13    postal_code VARCHAR(20),  
14    region VARCHAR(50),  
15    product_id VARCHAR(50),  
16    category VARCHAR(100),  
17    sub_category VARCHAR(100),  
18    product_name VARCHAR(255),  
19    sales NUMERIC,  
20    quantity INT,  
21    discount NUMERIC,  
22    profit NUMERIC  
23 );
```

Below the editor, a green status bar indicates the query was executed successfully, affecting 0 rows in 14.4ms. A message at the bottom states: 'Query executed successfully. No rows returned.'



The screenshot shows the same SQL IDE window, but now displaying an error. The top toolbar shows the dropdown menu set to 'SQL', a status indicator 'ERROR duplicate key value violates unique constraint "superstore_pkey" -- 7.3ms', a button 'import_data', a 'Run' button, and links for 'Prettify SQL', 'Wiki', and navigation arrows. The main editor contains a SQL script to copy data from a CSV file into the 'superstore' table:

```
1 COPY superstore  
2 FROM  
3     '/tmp/superstore.csv' DELIMITER ',' CSV HEADER ENCODING 'WIN1252';
```

Below the editor, a red status bar indicates an error occurred, taking 7.3ms. The error message is: 'ERROR 23505 duplicate key value violates unique constraint "superstore_pkey"'. The details of the error are as follows:

```
1 COPY superstore  
2 FROM  
3     '/tmp/superstore.csv' DELIMITER ',' CSV HEADER ENCODING 'WIN1252';  
  
DETAIL: Key (row_id)=(1) already exists.  
WHERE: COPY superstore, line 2  
SCHEMA: public  
TABLE: superstore  
CONSTRAINT: superstore_pkey
```

At the bottom of the window, there is a 'MARKDOWN' tab.

Step-2: Exploring:

SQL OK - 21 rows - 10.4ms schema info Run Prettify SQL Wiki ↑ ↓ ×

```
1 SELECT
2   column_name,
3   data_type,
4   character_maximum_length,
5   is_nullable
6 FROM
7   information_schema.columns
8 WHERE
9   table_name = 'superstore'
10 ORDER BY
11   ordinal_position;
```

21 rows · 10.4ms

☒	column_name	data_type	character_maximum_length	is_nullable
	name	varchar	int4	varchar
1	row_id	integer	NULL	NO
2	order_id	character varying	50	YES
3	order_date	character varying	50	YES
4	ship_date	character varying	50	YES
5	ship_mode	character varying	50	YES
6	customer_id	character varying	50	YES
7	customer_name	character varying	100	YES
8	segment	character varying	50	YES
9	country	character varying	100	YES
10	city	character varying	100	YES
11	state	character varying	100	YES
12	postal_code	character varying	20	YES

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 21 COLUMNS: 4 10.4ms CSV Excel

SQL OK - 1 row - 6.1ms row_count Run Prettify SQL Wiki ↑ ↓ ×

```
1 SELECT COUNT(*) AS total_rows FROM superstore;
```

1 rows · 6.1ms

☒	total_rows
	int8
1	9 994

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 1 COLUMNS: 1 6.1ms CSV Excel

SQL OK - 1 row - 112.5ms distinct_counts Run Prettify SQL Wiki ↑ ↓ ×

```
1 -- 2Distinct values for key categorical columns
2
3 SELECT
4   COUNT(DISTINCT order_id) AS unique_orders,
5   COUNT(DISTINCT customer_id) AS unique_customers,
6   COUNT(DISTINCT product_id) AS unique_products,
7   COUNT(DISTINCT category) AS unique_categories,
8   COUNT(DISTINCT sub_category) AS unique_sub_categories,
9   COUNT(DISTINCT region) AS unique_regions,
10  COUNT(DISTINCT state) AS unique_states,
11  COUNT(DISTINCT segment) AS unique_segments,
12  COUNT(DISTINCT ship_mode) AS unique_ship_modes
13 FROM superstore;
```

1 rows · 112.5ms

☒	unique_orders	unique_customers	unique_products	unique_categories	unique_sub_categories
	int8	int8	int8	int8	int8
1	5 009	793	1 862	3	17

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 1 COLUMNS: 9 112.5ms CSV Excel

SQL

OK - 1 row - 12.6ms

stats_summary

Run

Prettify SQL

Wiki

↑

↓

×

```

1  -- 2b. Quick statistical summary of numeric columns
2  SELECT
3      ROUND(MIN(sales), 2) AS min_sales,
4      ROUND(MAX(sales), 2) AS max_sales,
5      ROUND(AVG(sales), 2) AS avg_sales,
6      ROUND(MIN(profit), 2) AS min_profit,
7      ROUND(MAX(profit), 2) AS max_profit,
8      ROUND(AVG(profit), 2) AS avg_profit,
9      MIN(quantity) AS min_qty,
10     MAX(quantity) AS max_qty,
11     ROUND(AVG(quantity), 2) AS avg_qty,
12     ROUND(MIN(discount), 2) AS min_discount,
13     ROUND(MAX(discount), 2) AS max_discount,
14     ROUND(AVG(discount), 2) AS avg_discount
15 FROM
16     superstore;

```

1 rows · 12.6ms

☒	min_sales	max_sales	avg_sales	min_profit	max_profit	avg_profit	m
	numeric	numeric	numeric	numeric	numeric	numeric	int4
1	0.44	22 638.48	229.86	-6 599.98	8 399.98	28.66	1

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

▶

ROWS: 1

COLUMNS: 12

12.6ms

CSV

Excel

Step-3: Applying where filters

SQL

OK - 10 rows - 9.4ms

filter_region

Run

Prettify SQL

Wiki

↑

↓

×

```

1  -- 3a. Filter by Region: Orders from the 'West' region
2  SELECT
3      order_id,
4      customer_name,
5      city,
6      state,
7      sales,
8      profit
9  FROM
10     superstore
11 WHERE
12     region = 'West'
13 ORDER BY
14     sales DESC
15 LIMIT
16     10;

```

10 rows · 9.4ms

☒	order_id	customer_name	city	state	sales	profit
	varchar	varchar	varchar	varchar	numeric	numeric
1	CA-2017-140151	Raymond Buch	Seattle	Washington	13 999.96	6 719.98
2	CA-2014-143917	Ken Lonsdale	San Francisco	California	8 187.65	327.50
3	CA-2017-135909	Jane Waco	Sacramento	California	5 083.96	1 906.48
4	CA-2016-136301	Edward Hooks	San Francisco	California	4 912.59	196.50
5	CA-2017-149881	Nick Crebassa	San Francisco	California	4 799.98	359.99
6	CA-2016-138478	Dennis Pardue	North Las Vegas	Nevada	4 535.97	1 644.29
7	CA-2016-100300	Max Jones	San Diego	California	4 476.8	503.64
8	CA-2016-159016	Karen Ferguson	Los Angeles	California	4 158.91	363.90
9	CA-2014-168494	Nora Preis	Fresno	California	3 610.84	135.40
10	CA-2016-107104	Maribeth Schnelling	Los Angeles	California	3 406.66	160.31

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

▶

ROWS: 10

COLUMNS: 6

9.4ms

CSV

Excel

```
1  -- 3b. Filter by Category: Only 'Technology' products
2  SELECT
3      order_id,
4      product_name,
5      sub_category,
6      sales,
7      profit
8  FROM
9      superstore
10 WHERE
11     category = 'Technology'
12 ORDER BY
13     sales DESC
14 LIMIT
15     10;
```

10 rows · 7.5ms

	order_id varchar	product_name varchar	sub_category varchar	sales numeric
1	CA-2014-145317	Cisco TelePresence System EX90 Videoconferencing Unit	Machines	22 638.48
2	CA-2016-118689	Canon imageCLASS 2200 Advanced Copier	Copiers	17 499.95
3	CA-2017-140151	Canon imageCLASS 2200 Advanced Copier	Copiers	13 999.96
4	CA-2017-127180	Canon imageCLASS 2200 Advanced Copier	Copiers	11 199.96
5	CA-2017-166709	Canon imageCLASS 2200 Advanced Copier	Copiers	10 499.97
6	US-2016-107440	3D Systems Cube Printer, 2nd Generation, Magenta	Machines	9 099.93
7	CA-2016-158841	HP Designjet T520 Inkjet Large Format Printer - 24" Color	Machines	8 749.95
8	CA-2016-143714	Canon imageCLASS 2200 Advanced Copier	Copiers	8 399.97
9	CA-2014-139892	Lexmark MX611dhe Monochrome Laser Printer	Machines	8 159.95
10	US-2017-168116	Cubify CubeX 3D Printer Triple Head Print	Machines	7 999.98

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 10 COLUMNS: 5 7.5ms

CSV Excel

```
1  -- 3c. Filter by Date: Orders placed in 2017
2  SELECT
3      order_id,
4      order_date,
5      customer_name,
6      sales,
7      profit
8  FROM
9      superstore
10 WHERE
11     TO_DATE(order_date, 'MM/DD/YYYY') BETWEEN '2017-01-01' AND '2017-12-31'
12 ORDER BY
13     TO_DATE(order_date, 'MM/DD/YYYY')
14 LIMIT
15     15;
```

15 rows · 16.5ms

	order_id varchar	order_date varchar	customer_name varchar	sales numeric	profit numeric
1	CA-2017-151750	1/1/2017	Janet Martin	12.73	2.22
2	CA-2017-151750	1/1/2017	Janet Martin	310.74	-26.63
3	CA-2017-107503	1/1/2017	Guy Armstrong	48.89	8.55
4	CA-2017-151750	1/1/2017	Janet Martin	15.22	-38.82
5	CA-2017-151750	1/1/2017	Janet Martin	13.74	-22.68
6	CA-2017-151750	1/1/2017	Janet Martin	6.47	-9.70
7	CA-2017-151750	1/1/2017	Janet Martin	141.42	-187.38
8	CA-2017-154466	1/1/2017	Dennis Pardue	3.6	1.72
9	CA-2017-144463	1/1/2017	Steven Cartwright	474.43	199.26
10	CA-2017-151750	1/1/2017	Janet Martin	454.56	-107.95
11	CA-2017-113481	1/2/2017	Aaron Smayling	695.7	-27.82
12	CA-2017-113481	1/2/2017	Aaron Smayling	15.66	-12.52

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 15 COLUMNS: 5 16.5ms

CSV Excel

SQL

OK - 15 rows - 9.5msfilter_salesRunPrettify SQLWiki↑↓×

```
1  -- 3d. Filter by Sales: High-value orders (sales > 1000)
2  SELECT
3      order_id,
4      product_name,
5      category,
6      sales,
7      profit,
8      discount
9  FROM
10     superstore
11 WHERE
12     sales > 1000
13 ORDER BY
14     sales DESC
15 LIMIT
16     15;
```

15 rows - 9.5ms

	order_id varchar	product_name varchar	category varchar
1	CA-2014-145317	Cisco TelePresence System EX90 Videoconferencing Unit	Technology
2	CA-2016-118689	Canon imageCLASS 2200 Advanced Copier	Technology
3	CA-2017-140151	Canon imageCLASS 2200 Advanced Copier	Technology
4	CA-2017-127180	Canon imageCLASS 2200 Advanced Copier	Technology
5	CA-2017-166709	Canon imageCLASS 2200 Advanced Copier	Technology
6	CA-2016-117121	GBC Ibimaster 500 Manual ProClick Binding System	Office Suppli
7	CA-2014-116904	Ibico EPK-21 Electric Binding System	Office Suppli
8	US-2016-107440	3D Systems Cube Printer, 2nd Generation, Magenta	Technology
9	CA-2016-158841	HP Designjet T520 Inkjet Large Format Printer - 24" Color	Technology
10	CA-2016-143714	Canon imageCLASS 2200 Advanced Copier	Technology
11	CA-2014-143917	High Speed Automatic Electric Letter Opener	Office Suppli
12	CA-2014-139892	Lexmark MX611dhe Monochrome Laser Printer	Technology

Step-4: Using group by:

SQLOK - 3 rows - 13.2msgroup_categoryRunPrettify SQLWiki↑↓×

```
1 -- 4a. Total Sales, Profit, and Quantity by Category
2 SELECT
3     category,
4     ROUND(SUM(sales), 2) AS total_sales,
5     ROUND(SUM(profit), 2) AS total_profit,
6     SUM(quantity) AS total_quantity,
7     COUNT(*) AS order_count
8 FROM
9     superstore
10 GROUP BY
11     category
12 ORDER BY
13     total_sales DESC;
```

3 rows - 13.2ms

	category	total_sales	total_profit	total_quantity	order_count
1	Technology	836 154.03	145 454.95	6 939	1 847
2	Furniture	741 999.80	18 451.27	8 028	2 121
3	Office Supplies	719 047.03	122 490.80	22 906	6 026

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 3 COLUMNS: 5 13.2msCSVExcel

SQLOK - 17 rows - 12.8msgroup_subcategoryRunPrettify SQLWiki↑↓×

```
1 -- 4b. Sales and Profit by Sub-Category
2 SELECT
3     sub_category,
4     ROUND(SUM(sales), 2) AS total_sales,
5     ROUND(SUM(profit), 2) AS total_profit,
6     ROUND(AVG(discount), 2) AS avg_discount,
7     COUNT(*) AS order_count
8 FROM
9     superstore
10 GROUP BY
11     sub_category
12 ORDER BY
13     total_profit DESC;
```

17 rows - 12.8ms

	sub_category	total_sales	total_profit	avg_discount	order_count
1	Copiers	149 528.03	55 617.82	0.16	68
2	Phones	330 007.05	44 515.73	0.15	889
3	Accessories	167 380.32	41 936.64	0.08	775
4	Paper	78 479.21	34 053.57	0.07	1 370
5	Binders	203 412.73	30 221.76	0.37	1 523
6	Chairs	328 449.10	26 590.17	0.17	617
7	Storage	223 843.61	21 278.83	0.07	846
8	Appliances	107 532.16	18 138.01	0.17	466
9	Furnishings	91 705.16	13 059.14	0.14	957
10	Envelopes	16 476.40	6 964.18	0.08	254
11	Art	27 118.79	6 527.79	0.07	796
12	Labels	12 486.31	5 546.25	0.07	364

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 17 COLUMNS: 5 12.8msCSVExcel

SQL

OK - 4 rows - 57.2ms group_region

Run

Prettify SQL

Wiki

↑

↓

✕

```
1 -- 4c. Sales and Profit by Region
2 SELECT
3     region,
4     ROUND(SUM(sales), 2) AS total_sales,
5     ROUND(SUM(profit), 2) AS total_profit,
6     ROUND(AVG(sales), 2) AS avg_order_value,
7     SUM(quantity) AS total_quantity,
8     COUNT(DISTINCT order_id) AS unique_orders
9 FROM
10     superstore
11 GROUP BY
12     region
13 ORDER BY
14     total_sales DESC;
```

4 rows - 57.2ms

	region	total_sales	total_profit	avg_order_value	total_quantity	unique_orders
	varchar	numeric	numeric	numeric	int8	int8
1	West	725 457.82	108 418.45	226.49	12 266	1 611
2	East	678 781.24	91 522.78	238.34	10 618	1 401
3	Central	501 239.89	39 706.36	215.77	8 780	1 175
4	South	391 721.91	46 749.43	241.80	6 209	822

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 4 COLUMNS: 6 57.2ms

CSV

Excel

SQL

OK - 3 rows - 12.1ms group_segment

Run

Prettify SQL

Wiki

↑

↓

✕

```
1 -- 4d. Average Sales and Profit by Segment
2 SELECT
3     segment,
4     COUNT(*) AS order_count,
5     ROUND(AVG(sales), 2) AS avg_sales,
6     ROUND(AVG(profit), 2) AS avg_profit,
7     ROUND(AVG(quantity), 2) AS avg_quantity
8 FROM
9     superstore
10 GROUP BY
11     segment
12 ORDER BY
13     avg_sales DESC;
```

3 rows - 12.1ms

	segment	order_count	avg_sales	avg_profit	avg_quantity
	varchar	int8	numeric	numeric	numeric
1	Home Office	1 783	240.97	33.82	3.78
2	Corporate	3 020	233.82	30.46	3.84
3	Consumer	5 191	223.73	25.84	3.76

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 3 COLUMNS: 5 12.1ms

CSV

Excel

Step-5: Sorting and limiting:

SQL

OK -- 10 rows -- 16.6ms top_products_sal Run Prettify SQL Wiki ↑ ↓ ×

```
1  -- 5a. Top 10 Products by Total Sales
2  SELECT
3      product_name,
4      category,
5      sub_category,
6      ROUND(SUM(sales), 2) AS total_sales,
7      SUM(quantity) AS total_qty_sold
8  FROM
9      superstore
10 GROUP BY
11     product_name,
12     category,
13     sub_category
14 ORDER BY
15     total_sales DESC
16 LIMIT
17     10;
```

10 rows · 16.6ms

☒	product_name varchar	category varchar	sub_category varchar
1	Canon imageCLASS 2200 Advanced Copier	Technology	Copiers
2	Fellowes PB500 Electric Punch Plastic Comb Binding Machine with Manual Bind	Office Supplies	Binders
3	Cisco TelePresence System EX90 Videoconferencing Unit	Technology	Machines
4	HON 5400 Series Task Chairs for Big and Tall	Furniture	Chairs
5	GBC DocuBind TL300 Electric Binding System	Office Supplies	Binders
6	GBC Ibimaster 500 Manual ProClick Binding System	Office Supplies	Binders
7	Hewlett Packard LaserJet 3310 Copier	Technology	Copiers
8	HP Designjet T520 Inkjet Large Format Printer - 24" Color	Technology	Machines
9	GBC DocuBind P400 Electric Binding System	Office Supplies	Binders
10	High Speed Automatic Electric Letter Opener	Office Supplies	Supplies

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

▶

ROWS: 10 COLUMNS: 5 16.6ms

CSV Excel


```
1  -- 5b. Top 10 Most Profitable Products
2  SELECT
3      product_name,
4      category,
5      ROUND(SUM(profit), 2) AS total_profit,
6      ROUND(SUM(sales), 2) AS total_sales
7  FROM
8      superstore
9  GROUP BY
10     product_name,
11     category
12 ORDER BY
13     total_profit DESC
14 LIMIT
15     10;
```

10 rows · 17.4ms

☒	product_name varchar	category varchar	total_profit numeric
1	Canon imageCLASS 2200 Advanced Copier	Technology	25 199.93
2	Fellowes PB500 Electric Punch Plastic Comb Binding Machine with Manual Bind	Office Supplies	7 753.04
3	Hewlett Packard LaserJet 3310 Copier	Technology	6 983.88
4	Canon PC1060 Personal Laser Copier	Technology	4 570.93
5	HP Designjet T520 Inkjet Large Format Printer - 24" Color	Technology	4 094.98
6	Ativa V4110MDD Micro-Cut Shredder	Technology	3 772.95
7	3D Systems Cube Printer, 2nd Generation, Magenta	Technology	3 717.97
8	Plantronics Savi W720 Multi-Device Wireless Headset System	Technology	3 696.28
9	Ibico EPK-21 Electric Binding System	Office Supplies	3 345.28
10	Zebra ZM400 Thermal Label Printer	Technology	3 343.54

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 10 COLUMNS: 4 17.4ms

CSV

Excel

SQL

OK -- 10 rows - 22.1ms bottom_products Run Prettify SQL Wiki ↑ ↓ ×

```

1  -- 5c. Bottom 10 Products (Biggest Losses)
2  SELECT
3      product_name,
4      category,
5      sub_category,
6      ROUND(SUM(profit), 2) AS total_profit,
7      ROUND(SUM(sales), 2) AS total_sales,
8      ROUND(AVG(discount), 2) AS avg_discount
9  FROM
10     superstore
11  GROUP BY
12     product_name,
13     category,
14     sub_category
15  ORDER BY
16     total_profit ASC
17  LIMIT
18     10;

```

10 rows · 22.1ms

product_name	category	sub_category	total_profit
ubeX 3D Printer Double Head Print	Technology	Machines	-8 879.97
MX611dhe Monochrome Laser Printer	Technology	Machines	-4 589.97
ubeX 3D Printer Triple Head Print	Technology	Machines	-3 839.99
ft Bull-Nose Wood Oval Conference Tables & Bases	Furniture	Tables	-2 876.12
antage Collection Racetrack Conference Table	Furniture	Tables	-1 934.40
Bind P400 Electric Binding System	Office Supplies	Binders	-1 878.17
lePresence System EX90 Videoconferencing Unit	Technology	Machines	-1 811.08
ale Chadless Opener Electric Letter Opener	Office Supplies	Supplies	-1 299.18
id Wood Round Tables	Furniture	Tables	-1 201.06
e By Design Rectangular and Half-Moon Meeting Room Tables	Furniture	Tables	-1 148.44

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 10 COLUMNS: 6 22.1ms

CSV

Excel

Step-6: Solve use cases:

```
1  -- 6a. Monthly Sales Trend (across all years)
2  SELECT
3      EXTRACT(
4          YEAR
5          FROM
6          TO_DATE(order_date, 'MM/DD/YYYY')
7      ) AS YEAR,
8      EXTRACT(
9          MONTH
10         FROM
11         TO_DATE(order_date, 'MM/DD/YYYY')
12     ) AS MONTH,
13     ROUND(SUM(sales), 2) AS total_sales,
14     ROUND(SUM(profit), 2) AS total_profit,
15     COUNT(DISTINCT order_id) AS order_count
16 FROM
17     superstore
18 GROUP BY
19     YEAR,
20     MONTH
21 ORDER BY
22     YEAR,
23     MONTH;
```

48 rows - 71.8ms

	year numeric	month numeric	total_sales numeric	total_profit numeric	order_count int8
1	2 014	1	14 236.90	2 450.19	32
2	2 014	2	4 519.89	862.31	28
3	2 014	3	55 691.01	498.73	71
4	2 014	4	28 295.35	3 488.84	66
5	2 014	5	23 648.29	2 738.71	69
6	2 014	6	34 595.13	4 976.52	66
7	2 014	7	33 946.39	-841.48	65
8	2 014	8	27 909.47	5 318.11	72
9	2 014	9	81 777.35	8 328.10	130
10	2 014	10	31 453.39	3 448.26	78
11	2 014	11	78 628.72	9 292.13	151
12	2 014	12	69 545.62	8 983.57	141

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

```
1  -- 6b. Yearly Sales Summary
2  SELECT
3      EXTRACT(
4          YEAR
5          FROM
6          TO_DATE(order_date, 'MM/DD/YYYY')
7      ) AS YEAR,
8      ROUND(SUM(sales), 2) AS total_sales,
9      ROUND(SUM(profit), 2) AS total_profit,
10     SUM(quantity) AS total_quantity,
11     COUNT(DISTINCT order_id) AS total_orders,
12     COUNT(DISTINCT customer_id) AS total_customers
13 FROM
14     superstore
15 GROUP BY
16     YEAR
17 ORDER BY
18     YEAR;
```

4 rows - 84.5ms

	year numeric	total_sales numeric	total_profit numeric	total_quantity int8	total_orders int8	total_customers int8
1	2 014	484 247.50	49 543.97	7 581	969	595
2	2 015	470 532.51	61 618.60	7 979	1 038	573
3	2 016	609 205.60	81 795.17	9 837	1 315	638
4	2 017	733 215.26	93 439.27	12 476	1 687	693

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 4 COLUMNS: 6 84.5ms

CSV Excel

SQL OK - 10 rows - 36.3ms top_customers Run Prettify SQL Wiki ↑ ↓ ×

```

1  -- 6c. Top 10 Customers by Total Spend
2  SELECT
3      customer_id,
4      customer_name,
5      segment,
6      ROUND(SUM(sales), 2) AS total_spent,
7      ROUND(SUM(profit), 2) AS total_profit,
8      COUNT(DISTINCT order_id) AS total_orders,
9      SUM(quantity) AS items_purchased
10 FROM
11     superstore
12 GROUP BY
13     customer_id,
14     customer_name,
15     segment
16 ORDER BY
17     total_spent DESC
18 LIMIT
19     10;

```

10 rows · 36.3ms

	customer_id	customer_name	segment	total_spent	total_profit	total_orders
	varchar	varchar	varchar	numeric	numeric	int8
1	SM-20320	Sean Miller	Home Office	25 043.05	-1 980.74	5
2	TC-20980	Tamara Chand	Corporate	19 052.22	8 981.32	5
3	RB-19360	Raymond Buch	Consumer	15 117.34	6 976.10	6
4	TA-21385	Tom Ashbrook	Home Office	14 595.62	4 703.79	4
5	AB-10105	Adrian Barton	Consumer	14 473.57	5 444.81	10
6	KL-16645	Ken Lonsdale	Consumer	14 175.23	806.86	12
7	SC-20095	Sanjit Chand	Consumer	14 142.33	5 757.41	9
8	HL-15040	Hunter Lopez	Consumer	12 873.30	5 622.43	6
9	SE-20110	Sanjit Engle	Consumer	12 209.44	2 650.68	11
10	CC-12370	Christopher Conant	Consumer	12 129.07	2 177.05	5

SQL OK - 49 rows - 43.0ms repeat_customer: Run Prettify SQL Wiki ↑ ↓ ×

```

1  -- 6d. Repeat Customers (more than 10 orders)
2  SELECT
3      customer_id,
4      customer_name,
5      segment,
6      COUNT(DISTINCT order_id) AS order_count,
7      ROUND(SUM(sales), 2) AS total_sales
8  FROM
9      superstore
10 GROUP BY
11     customer_id,
12     customer_name,
13     segment
14 HAVING
15     COUNT(DISTINCT order_id) > 10
16 ORDER BY
17     order_count DESC;

```

49 rows · 43.0ms

	customer_id	customer_name	segment	order_count	total_sales
	varchar	varchar	varchar	int8	numeric
1	EP-13915	Emily Phan	Consumer	17	5 478.06
2	NS-18640	Noel Staavos	Corporate	13	2 964.82
3	SH-19975	Sally Hughsby	Corporate	13	3 406.84
4	JE-15745	Joel Eaton	Consumer	13	6 760.82
5	CK-12205	Chloris Kastensmidt	Consumer	13	3 154.86
6	EA-14035	Erin Ashbrook	Corporate	13	2 846.71
7	ZC-21910	Zuschuss Carroll	Consumer	13	8 025.71
8	PG-18820	Patrick Gardner	Consumer	13	3 086.91
9	DK-12835	Damala Kotsonis	Corporate	12	3 256.48
10	BD-11320	Bill Donatelli	Consumer	12	5 718.52
11	BP-11095	Bart Pistole	Corporate	12	2 442.04
12	CC-12220	Chris Cortes	Consumer	12	3 913.42

SQL

OK -- 0 rows -- 6.9ms duplicate_rows

Run

Prettify SQL

Wiki

↑

↓

×

```

1  -- 6e. Check for Duplicate Rows (same row_id appearing more than once)
2  SELECT
3      row_id,
4      COUNT(*) AS occurrence
5  FROM
6      superstore
7  GROUP BY
8      row_id
9  HAVING
10     COUNT(*) > 1
11 ORDER BY
12     occurrence DESC;

```

-- 6e. Check for Duplicate Rows (same row_id appearing more than once) SELECT row_id, COUNT(*) AS occurrence FROM superstore GROUP BY row_id HAVING COUNT(*) > 1 ORDER BY occurrence DESC;
 0 rows affected • 6.9ms

Query executed successfully. No rows returned.

SQL

OK -- 8 rows -- 11.1ms duplicate_order!

Run

Prettify SQL

Wiki

↑

↓

×

```

1  -- 6f. Check for Duplicate Orders (same order with same product)
2  SELECT
3      order_id,
4      product_id,
5      customer_id,
6      COUNT(*) AS occurrence
7  FROM
8      superstore
9  GROUP BY
10     order_id,
11     product_id,
12     customer_id
13 HAVING
14     COUNT(*) > 1
15 ORDER BY
16     occurrence DESC
17 LIMIT
18     10;

```

8 rows - 11.1ms

	order_id varchar	product_id varchar	customer_id varchar	occurrence int8
1	CA-2017-152912	OFF-ST-10003208	BM-11650	2
2	CA-2016-137043	FUR-FU-10003664	LC-17140	2
3	CA-2016-129714	OFF-PA-10001970	AB-10060	2
4	US-2014-150119	FUR-CH-10002965	LB-16795	2
5	US-2016-123750	TEC-AC-10004659	RB-19795	2
6	CA-2016-140571	OFF-PA-10001954	SJ-20125	2
7	CA-2017-118017	TEC-AC-10002006	LC-16870	2
8	CA-2015-103135	OFF-BI-10000069	SS-20515	2

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 8 COLUMNS: 4 11.1ms

CSV

Excel

Step-7: Validating results:

SQL OK - 1 row - 6.6ms validate_count Run Prettify SQL Wiki ↑ ↓ ×

```
1 -- 7a. Total row count (should be 9994 for the standard Superstore dataset)
2 SELECT
3     COUNT(*) AS total_rows
4 FROM
5     superstore;
```

1 rows - 6.6ms

	total_rows int8
1	9 994

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 1 COLUMNS: 1 6.6ms CSV Excel

SQL OK - 1 row - 10.9ms null_check Run Prettify SQL Wiki ↑ ↓ ×

```
1 -- 7b. Check for NULL values in critical columns
2 SELECT
3     SUM(
4         CASE
5             WHEN order_id IS NULL THEN 1
6             ELSE 0
7         END
8     ) AS null_order_id,
9     SUM(
10        CASE
11            WHEN customer_id IS NULL THEN 1
12            ELSE 0
13        END
14    ) AS null_customer_id,
15    SUM(
16        CASE
17            WHEN product_id IS NULL THEN 1
18            ELSE 0
19        END
20    ) AS null_product_id,
21    SUM(
22        CASE
23            WHEN sales IS NULL THEN 1
24            ELSE 0
25        END
26    ) AS null_sales,
27    SUM(
28        CASE
29            WHEN profit IS NULL THEN 1
30            ELSE 0
31        END
32    ) AS null_profit,
33    SUM(
34        CASE
35            WHEN category IS NULL THEN 1
36            ELSE 0
37        END
38    ) AS null_category,
39    SUM(
40        CASE
41            WHEN region IS NULL THEN 1
42            ELSE 0
43        END
44    ) AS null_region
45 FROM
46     superstore;
```

1 rows - 10.9ms

	null_order_id int8	null_customer_id int8	null_product_id int8	null_sales int8	null_profit int8	null_cate int8
1	0	0	0	0	0	0

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 1 COLUMNS: 7 10.9ms CSV Excel

SQL OK - 1 row - 29.5ms cross verify Run Prettify SQL Wiki ↑ ↓ ×

```

4  ROUND(
5  (
6  | SELECT
7  |   SUM(sales)
8  | FROM
9  |   superstore
10 | WHERE
11 |   region = 'East'
12 | ) + (
13 | SELECT
14 |   SUM(sales)
15 | FROM
16 |   superstore
17 | WHERE
18 |   region = 'West'
19 | ) + (
20 | SELECT
21 |   SUM(sales)
22 | FROM
23 |   superstore
24 | WHERE
25 |   region = 'Central'
26 | ) + (
27 | SELECT
28 |   SUM(sales)
29 | FROM
30 |   superstore
31 | WHERE
32 |   region = 'South'
33 | ),
34 | 2
35 | ) AS sum_of_regions,
36 CASE
37 | WHEN ROUND(SUM(sales), 2) = ROUND(
38 | (
39 | | SELECT
40 | |   SUM(sales)
41 | | FROM
42 | |   superstore
43 | | WHERE
44 | |   region = 'East'
45 | | ) + (
46 | | SELECT
47 | |   SUM(sales)
48 | | FROM
49 | |   superstore
50 | | WHERE
51 | |   region = 'West'
52 | | ) + (
53 | | SELECT
54 | |   SUM(sales)
55 | | FROM

```

1 rows - 29.5ms

	total_sales numeric	sum_of_regions numeric	validation_result text
1	2 297 200.86	2 297 200.86	PASS

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

SQL OK - 1 row - 13.9ms date_range Run Prettify SQL Wiki ↑ ×

```

1  -- 7d. Date range validation
2  SELECT
3  | MIN(TO_DATE(order_date, 'MM/DD/YYYY')) AS earliest_order,
4  | MAX(TO_DATE(order_date, 'MM/DD/YYYY')) AS latest_order,
5  | MAX(TO_DATE(order_date, 'MM/DD/YYYY')) - MIN(TO_DATE(order_date, 'MM/DD/YYYY')) AS date_span_days
6  FROM
7  | superstore;

```

1 rows - 13.9ms

	earliest_order date	latest_order date	date_span_days int4
1	2014-01-02T18:30:00.000Z	2017-12-29T18:30:00.000Z	1 457

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 1 COLUMNS: 3 13.9ms

CSV Excel